An Exact Snoop Filter for Seamless Cache Coherence in FPGA-Based Multi-Core RISC-V

Anonymous submission

***Abstract*—Attaching a private, snoop-coherent L1 data cache to an unmodified RISC-V core is an attractive way to build FPGA multi-core systems, and a recently proposed design does so with coherence handled entirely in hardware. We reproduce that design in open-source RTL and measure where its time actually goes. Across four benchmark kernels the cluster issues 1,048,821 invalidation broadcasts and not one of them invalidates a line in another cache. Every kernel partitions its data per processing element, so every written line is private, yet each write still pays a cluster-wide arbitration and a handshake that waits on every other cache. That accounts for between 6 and 33% of a core's execution time, and on a streaming kernel it makes the cache extension slower than having no cache at all. We add a snoop filter that keeps an exact mirror of the caches' tag arrays and suppresses a broadcast when no other cache holds the line. It costs 1,009 LUTs and 519 flip-flops, preserves coherence under a version-monotonicity checker, and is worth a geometric mean $1.12\times$, up to $1.22\times$ on matrix multiplication. It also converts the streaming kernel from $0.98\times$ to $1.10\times$ against an uncached baseline. We report the frequency it costs, and evaluate at the original design's own 60 MHz operating point.**

***Index Terms*—RISC-V, cache coherence, snoop filter, FPGA, multi-core**

I. Introduction

A soft multi-core system on an FPGA gains a great deal from being able to attach a data cache to an existing, verified RISC-V core without modifying it. The core stays an unmodified IP block, coherence is handled in hardware, and software needs no knowledge of either. Kamaleldin *et al.* [1] propose exactly this: a private write-through L1 data cache unit (DCU) attached through the core's ordinary memory interface, with a snooping invalidation bus maintaining coherence across the cluster.

We built that design from scratch in open-source SystemVerilog in order to measure it, and the measurement is the origin of this paper. The DCU exports per-access cycle accounting, and it says that between 6 and 33% of a core's time is spent waiting on the coherence bus. That is a large number for a mechanism that, on these workloads, turns out to accomplish nothing at all: across four kernels the cluster sends 1,048,821 invalidation broadcasts and none of them invalidates a line anywhere.

The reason is structural rather than accidental. The kernels are data-parallel and partition their arrays per processing element, so almost every written line is private to the core writing it. The bus has no way to know that, so every write is announced to every cache, and the announcement is serialised: at most one invalidation is granted per cycle, and the grant is withheld until every other cache is ready to accept the broadcast.

Our contributions are:

- A measurement of how much of this design's coherence traffic is necessary. On its own benchmark kernels, none of it is, and we quantify what that costs (Section III).
- An exact snoop filter: a structural mirror of the caches' tag arrays that lets the bus suppress a broadcast when no other cache holds the line. Being exact rather than conservative makes the correctness argument a paragraph (Section IV).
- An evaluation in cycles, in coherence correctness, and in place-and-routed FPGA cost, including the frequency the mechanism costs and the operating point at which it pays (Section V).
- Three mechanisms we designed, measured and rejected before this one, including one that was incorrect while passing every functional test it had (Section VI).

II. The Design Under Study

The DCU is a private, write-through, no-allocate L1 data cache sitting between an in-order RISC-V core and the memory system. Because writes go through to memory, shared memory always holds the current value and no dirty-line write-back or ownership state is needed. Coherence is maintained by an invalidation bus: a core write reaches the bus as an invalidation request, and on grant the address is broadcast to every other DCU, which drops any matching line.

Two properties of that bus matter here. Plain read requests are not arbitrated against each other – multiple readers proceed in the same cycle, which is the multiple-readers half of the MRSW invariant – but a read is withheld while another core has an invalidation outstanding for the same line, which is the single-writer half. Invalidations *are* arbitrated: at most one is granted per cycle across the cluster, and the grant is held back until every other DCU can accept the broadcast, so that no invalidation is ever lost.

Our reproduction targets the stated configuration: four processing elements, each an unmodified CV32E40P with a private 2 KiB two-way DCU of 16-byte lines, meeting shared memories through an AXI4 crossbar. A non-coherent baseline is built from the same sources with one parameter changed, so any comparison between them is attributable to the cache sub-system alone.

TABLE I
TIME SPENT WAITING FOR A SNOOPY-BUS GRANT, AS A PERCENTAGE OF CYCLES WITH AN ACCESS IN THE DCU.

Kernel	MemLat 2	MemLat 8	MemLat 20
memcpy (16 KiB)	14.3	26.1	**33.0**
matmul (64^2)	15.8	26.3	32.9
conv2d (64^2)	6.5	11.2	19.0
FFT (N=256)	5.8	8.4	12.2

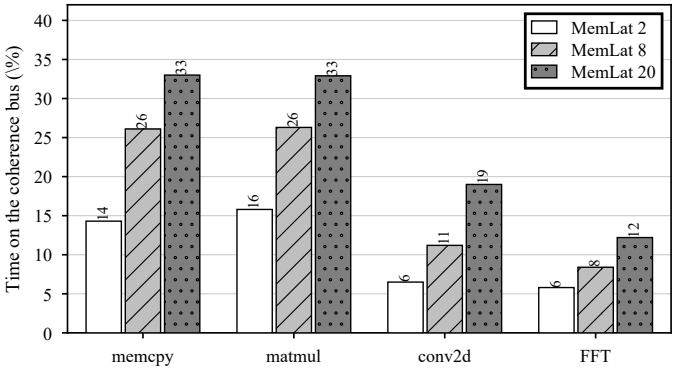


Fig. 1. Time each core spends waiting for a snoopy-bus grant, as a percentage of the cycles it had an access in the DCU. Table II shows that none of the traffic it is waiting for is necessary.

III. WHERE THE TIME GOES

A. The coherence bus is a large fraction of runtime

The DCU separates a core access into the places it can wait. One of those is stage 1 waiting for a snoopy-bus grant. Table I gives that bucket as a fraction of the cycles a core spent with an access in the DCU at all, and Fig. 1 plots it.

Between a twentieth and a third of a core's time goes to arbitrating for a resource that grants one request per cycle across the whole cluster. The share grows with memory latency, because each grant is held longer.

B. It is not readers waiting on writers

The obvious reading of Table I is contention between readers and writers – the single-writer screening doing its job. We instrumented the bus to count it, and across all four kernels a read is *never* withheld: not once. Whatever the stall is, it is not the MRSW screening.

By elimination it is the invalidations, which are the only arbitrated traffic left.

C. None of the invalidations are necessary

Each DCU therefore counts the broadcasts it receives and how many find a line to clear. Table II gives the result.

Over a million broadcasts, none of which invalidated anything. Each one paid a cluster-wide arbitration and a four-way handshake to inform nobody of anything.

A result this uniform deserves suspicion of the instrument rather than of the design, so the counter is controlled. Our coherence testbench shares deliberately, with a producer/consumer ping-pong and a randomised four-core stress

TABLE II
INVALIDATION BROADCASTS RECEIVED ACROSS THE CLUSTER, AND HOW MANY INVALIDATED A LINE. THE LAST ROW IS A CONTROL.

Workload	Broadcasts	Useful	%
matmul (64^2)	823,338	0	0.00
conv2d (64^2)	139,182	0	0.00
FFT (N=256)	61,683	0	0.00
memcpy (16 KiB)	24,618	0	0.00
sharing testbench	921	345	37.46

over a small window; on that workload the same counter reports 345 useful broadcasts of 921. The counter can see a useful invalidation. The zeroes are a property of the workloads.

They are a property worth stating plainly: these kernels are data-parallel and partition their data per core, so essentially every written line is private. That is not a weakness of the benchmark selection, because these are the kernels the original evaluation itself uses.

IV. AN EXACT SNOOP FILTER

A. Structure

The bus needs to answer one question before granting an invalidation: does any other cache hold this line? We answer it exactly, with a mirror of the caches' tag and valid arrays held in the bus and updated by the same two events that change the caches themselves – a refill installs a tag into a way, an invalidation clears one. Each DCU exports those events on a small port driven from the same signals that write its own arrays.

Exactness is therefore structural rather than argued. The mirror is written from the same signals, on the same clock edge, under the same conditions as the state it mirrors, so it cannot drift from it.

We rejected a hashed filter deliberately. A Bloom-style structure may only ever over-approximate: claiming a line is held when it is not costs a redundant broadcast and is safe, while the converse loses an invalidation and breaks coherence. Insert-only hashed filters are safe but saturate, and matmul touches far more lines than an affordable bitmap has bits; removing from one requires knowing when a line leaves a cache, which is exactly the information a mirror already has.

B. Why suppression is safe

When the mirror reports that no other cache holds the line, no other cache holds it, so a broadcast would clear nothing anywhere and not sending it is unobservable. That is the whole argument, and it is short because the structure is exact. A conservative filter would need an argument about the direction of its error; this one does not.

Everything else is untouched. Invalidations that *are* needed are still arbitrated, broadcast and acknowledged exactly as before; the single-writer screening on reads is unchanged; and the write-through lock that stops a remote read fetching a pre-write value is unchanged, because it protects against a value in memory rather than against a cached copy.

TABLE III
EXECUTION CYCLES OF THE SLOWEST PROCESSING ELEMENT. THE ORACLE SUPPRESSES EVERY BROADCAST AND BOUNDS WHAT A FILTER CAN ACHIEVE.

Kernel	Baseline	Filtered	Oracle	Speedup
matmul (64^2)	1,337,850	1,094,750	1,094,750	**1.222**
memcpy (16 KiB)	21,010	18,708	18,708	1.123
conv2d (64^2)	168,177	154,382	154,382	1.089
FFT (N=256)	85,093	80,228	80,228	1.061
Geometric mean				**1.122**

TABLE IV
POST-IMPLEMENTATION COST OF THE FILTER, XCZU7EV AT 60 MHZ. BOTH CONFIGURATIONS MEET ALL TIMING CONSTRAINTS.

	Filtered	Reference	Delta
LUT	27,776	26,767	+1,009
Flip-flops	15,673	15,154	+519
Block RAM tiles	124	124	0
DSP	20	20	0
WNS setup (ns)	+4.307	+2.597	
Failing endpoints	0	0	0

C. Implementation

The mirror is one array per (core, way) pair, each with a single write port and a single read port, addressed by set index – eight arrays at the geometry studied here. Tags are held outside any reset, since a tag is only compared when its valid bit says it means something, which lets them map to distributed RAM. Valid bits stay in flip-flops, where they can be reset and read eight at a time, at 512 bits rather than 11,264. This is the same division the DCU makes internally for the same reason.

That structure matters more than it might appear: a first implementation held tags and valid bits together under an asynchronous reset and read them combinationally, which Vivado mapped entirely to flip-flops – 11,776 of them, and 3,202 LUTs, against 512 and 414 for the structure above.

V. EVALUATION

A. Cycles

Table III and Fig. 2 give execution time with and without the filter, and against an oracle that suppresses every broadcast unconditionally – which is sound to simulate only because Table II says none of them are needed, and which bounds what any filter could return.

The filter reaches the oracle on every kernel, cycle for cycle. That is what an exact structure should do on workloads with no sharing: it suppresses precisely the broadcasts the oracle does, for a provable reason rather than an assumed one.

The snoop-stall bucket confirms the mechanism rather than merely the outcome. On conv2d it falls from 20,848 cycles to 109; on memcpy, from 7,168 to 1. FFT is the exception and falls only from 6,521 to 2,955, because load-reserved and store-conditional still pass the second arbiter whatever the broadcasts do – a residual cost no filter can remove.

B. The extension stops being harmful

The reproduction's most awkward finding was that on memcpy at low memory latency, the cache extension is *slower* than having no cache at all: $0.975\times$ the non-coherent baseline. With the filter it runs at $1.095\times$. The pathology is not intrinsic to attaching a coherent cache; it is the cost of announcing private writes to a cluster that does not care.

C. Coherence

The four kernels never exercise the filter's must-broadcast path, so they cannot establish that it is safe. Our coherence testbench does: it shares deliberately, and roughly half of its invalidations are useful. With the filter enabled it passes 3,112 checks, including a version-monotonicity invariant – a core that has observed version k of a location may never afterwards observe a version below k, which is exactly what a lost invalidation causes – and a final quiesce in which every core re-reads the whole window so that any line left stale in any cache is caught against memory. The filter suppressed 282 of 921 broadcasts and delivered every one that mattered.

D. FPGA cost

Both configurations were implemented on a Zynq UltraScale+ XCZU7EV in a non-project Vivado 2025.1 flow. Table IV gives the cost at 60 MHz, where both meet every constraint.

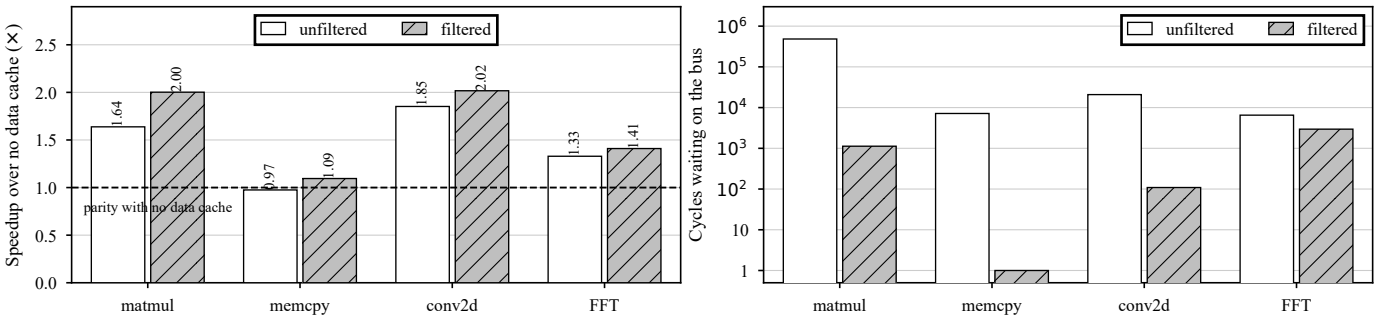
The filter costs 1,009 LUTs and 519 flip-flops, under 4% of the design, and no block RAM or DSP at all.

E. The frequency it costs

It does cost frequency, and the paper would be misleading without saying so. At a 75 MHz target the reference meets timing with $+1.327$ ns of slack and the filtered design does not, at -2.586 ns. The critical path is 86% routing and runs from a DCU's registered request address across the die; the filter lengthens it because the decision to broadcast is a cluster-spanning combinational path, and it does so however the mirror is built. Making the mirror smaller did not help: the flip-flop version above is four times the area and closes *better*, at -0.423 ns.

We therefore evaluate at 60 MHz, which both configurations meet comfortably, and which is the frequency the original design itself runs at rather than one chosen for convenience. At a fixed 60 MHz clock the filter's $1.122\times$ in cycles is a $1.122\times$ in wall time. Clocked freely, the reference reaches a higher frequency and the advantage narrows.

We deliberately do not report a maximum-frequency comparison from the 60 MHz builds. Measured at a constraint that is already met, that figure records where the tool stopped optimising rather than what the design can do: at 60 MHz it favours the filtered design by 9.8 MHz, and at 75 MHz it favours the reference by 20.5 MHz.



(a) what the extension is worth

(b) why

Fig. 2. (a) What the cache extension is worth against having no data cache at all, with and without the filter. memcpy begins below parity – the extension costs more than it saves – and the filter carries it back above. (b) Cycles spent waiting on the coherence bus, on a logarithmic scale, which is where the speedup in (a) comes from. FFT is the exception: its load-reserved and store-conditional traffic still passes the second arbiter, so its stall falls by half rather than to nothing.

VI. What Did Not Work

Three mechanisms preceded this one. We report them because each was rejected by a measurement that also sharpened the eventual design.

A streaming mode that would skip allocation and read at word granularity for phases with no reuse. The non-coherent baseline already is such a mode, and bounds it: on memcpy it beats the cache at a memory latency of two by 2.5% and loses by better than two to one at eight and twenty, because a line fill amortises one memory latency across four words.

A hit-speculative bypass, in which a read presumes it will hit and takes no bus transaction, and only a miss arbitrates. It was both useless and incorrect. Useless because plain reads were never arbitrated in the first place, so there was no cost to remove – it measured slower. Incorrect because the screening it removed is not about allocation but is the single-writer half of MRSW: a read that hits a resident line while a remote invalidation for that line is outstanding returns stale data. It passed all eleven functional checks of the benchmark harness, which is the point worth reporting: the race window is narrow and a green test said nothing about it.

Two structural fixes to the mirror, described in Section V: neither the register count nor the storage type was the cause of the frequency loss.

VII. Related Work

Filtering snoops is not a new idea, and the value here is not the concept but where the filter sits, what it is exact about, and what it is measured against.

JETTY [2] places a small cache-like structure between the bus and each processor's L2, filtering snoops that would miss locally so that the energy of a tag lookup is not spent. Its target is energy, its position is the receiving side, and it is approximate: a small JETTY filters most useless snoops, not all. Coarse-grain coherence tracking [3] and RegionScout [4] raise the granularity instead, tracking sharing over regions of hundreds or thousands of bytes so that a region known to be unshared needs no broadcast at all, trading precision for a much smaller structure.

Our setting inverts the cost that matters. On a snooping bus in a soft multi-core, the expense of an unnecessary invalidation is not the energy of the remote tag lookups but the *serialisation*: one grant per cycle across the cluster, held until every other cache is ready to accept the broadcast. The filter therefore belongs at the source, deciding whether to arbitrate at all, rather than at each receiver deciding whether to look. And because the design is small – four caches of 128 lines – an exact mirror costs 1,009 LUTs, which removes the need to trade precision for size and reduces the correctness argument to a sentence. A region-based scheme would be the right choice at a scale where an exact mirror is not affordable; at this one it is not necessary.

The RISC-V ecosystem offers coherent fabrics at a range of scales: TileLink [5] as a coherent interconnect standard, OpenPiton [6] scaling directory coherence to many cores, and CIFER [7] integrating coherent clusters in silicon. These are complete memory systems, and adopting one means adopting its interconnect. The design studied here occupies a deliberately different point: an extension attached to an unmodified core through its existing memory port, which is what makes it attractive on an FPGA and also what confines coherence to a single shared bus. Our contribution is to that design point rather than to coherent fabrics generally.

Finally, the measurement itself has a precedent worth acknowledging. JETTY's premise is the observation that many snoops find no copy elsewhere; we find that on the benchmark kernels of the design we study, *no* invalidation finds a copy elsewhere. The difference between "many" and "all" is a property of data-parallel workloads on a private-cache cluster, and it is what makes an exact filter worth its area here.

VIII. Threats to Validity

The central measurement is that none of the invalidation traffic is useful, and that is a property of workloads as much as of the design. Data-parallel kernels that partition per core have almost nothing to share. We claim only what we measured: on the kernels the original evaluation itself uses, this design's coherence traffic is entirely avoidable. A workload

with genuine sharing would see less benefit, and our own coherence testbench, at roughly half its invalidations useful, is the extreme case in the other direction.

The evaluation is in cycles from RTL simulation and in post-place-and-route resource and timing figures; no bitstream has been run on hardware. The original targets a Virtex UltraScale+ XCVU9P and this work an XCZU7EV, so absolute figures are directional rather than a like-for-like comparison.

Power is not reported. Vivado's vectorless estimate models this design as idle because it cannot evaluate the cores' clock-gate enables, and a switching-activity-based estimate is not yet complete.

IX. Conclusion

A seamless coherent cache extension for soft RISC-V multicores spends between 6 and 33% of each core's time on a coherence protocol that, on its own benchmark kernels, accomplishes nothing: over a million invalidation broadcasts, none of which invalidate anything. An exact mirror of the caches' tag arrays lets the bus suppress those broadcasts for 1,009 LUTs and 519 flip-flops, preserving coherence structurally rather than by argument, and is worth a geometric mean $1.122\times$ at a fixed 60 MHz clock. It also removes the case in which the extension was slower than having no cache at all. The mechanism costs frequency, and we report where that matters.

References

[1] A. Kamaleldin, M. Nickel, S. Wu and D. Göhringer, "Seamless Cache Extension for FPGA-based Multi-Core RISC-V SoC," in *Proc. IEEE 37th Int. System-on-Chip Conf. (SOCC)*, 2024.

[2] A. Moshovos, G. Memik, B. Falsafi and A. Choudhary, "JETTY: Filtering Snoops for Reduced Energy Consumption in SMP Servers," in *Proc. 7th Int. Symp. High-Performance Computer Architecture (HPCA)*, 2001.

[3] J. F. Cantin, M. H. Lipasti and J. E. Smith, "Improving Multiprocessor Performance with Coarse-Grain Coherence Tracking," in *Proc. 32nd Int. Symp. Computer Architecture (ISCA)*, 2005, pp. 246–257.

[4] A. Moshovos, "RegionScout: Exploiting Coarse Grain Sharing in Snoop-Based Coherence," in *Proc. 32nd Int. Symp. Computer Architecture (ISCA)*, 2005, pp. 234–245.

[5] W. Terpstra, "TileLink: A Free and Open-Source, High-Performance Scalable Cache-Coherent Fabric Designed for RISC-V," in *RISC-V Workshop*, 2017.

[6] J. Balkind *et al.*, "OpenPiton+Ariane: The First Open-Source, SMP Linux-booting RISC-V System Scaling from One to Many Cores," in *Workshop on Computer Architecture Research with RISC-V (CARRV)*, 2019.

[7] A. Li *et al.*, "CIFER: A Cache-Coherent 12nm 16mm^2 SoC with Four 64-Bit RISC-V Application Cores, 18 32-Bit RISC-V Compute Cores, and a 1541 LUT6/mm^2 Synthesizable eFPGA," in *IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2023.